cursor-agent-team: A Multi-Role, Single-Conversation Framework for Human-AI Collaboration

Kuang Hu
Yunnan University
Kunming, China
hukuang@ynu.edu.cn

Abstract

The rapid advancement of Large Language Models (LLMs) has catalyzed the development of Multi-Agent Systems (MAS) for complex problem-solving. However, conventional MAS architectures often suffer from context loss and identity fragmentation during agent handoffs, where the transfer of state between specialized agents degrades reasoning fidelity. We present `cursor-agent-team`, a framework that reimagines human-AI collaboration through a multi-role, single-conversation architecture. By eliminating distinct agent instances in favor of role-switching within a unified context window, we aim to reduce handoff overhead—a design goal we term "Minimal Handoff." Our methodology integrates Aspect-Oriented Programming (AOP) principles to manage cross-cutting concerns—such as persona consistency and knowledge gleaning—without polluting the core logic. We also employ a Spec-Script loop where natural language specifications drive deterministic Python script execution, ensuring reliability. Implemented within the Cursor IDE, this framework illustrates how Intelligence Augmentation (IA) can be realized not by replacing humans, but by orchestrating AI capabilities as a cohesive team within a single interaction stream.

1 Introduction

The vision of Intelligence Augmentation (IA), as originally articulated by Licklider [1960], posits a symbiotic partnership where computers handle routineizable tasks while humans provide strategic direction. In the era of Large Language Models (LLMs), this vision is often pursued through Multi-Agent Systems (MAS), where specialized agents collaborate to solve complex problems [Wu et al., 2023, Hong et al., 2023]. While effective for decomposition, MAS architectures introduce a critical bottleneck: the "handoff" problem. As tasks pass between agents, context is often compressed or lost, leading to degradation in reasoning quality and coherence, a phenomenon akin to the "Lost in the Middle" effect [Liu et al., 2023]. Current frameworks often prioritize agent capability over orchestration, resulting in systems where agents talk *at* each other rather than thinking *with* the human user [Shekkizhar et al., 2025].

We address this gap by proposing a methodology-first approach that prioritizes context continuity and explicit orchestration. We introduce `cursor-agent-team`, a framework built on the premise of a single, continuous conversation where the LLM adopts different "masks" or roles rather than handing off tasks to separate entities. This *Multi-Role, Single-Conversation* architecture eliminates the handoff friction entirely, ensuring that the full history of the problem-solving process remains immediately accessible.

To manage the complexity of this unified context, we adapt principles from Aspect-Oriented Programming (AOP) [Kiczales et al., 2001]. Cross-cutting concerns—such as maintaining a con-

sistent persona, logging insights (gleaning), or managing voice output—are handled as distinct "aspects" that are woven into the execution flow, keeping the core work logic clean. Additionally, we enforce rigorous execution through a Spec-Script integration, inspired by the "Blueprint First" philosophy [Qiu et al., 2025], where high-level role specifications drive the invocation of deterministic local scripts.

Our contributions are as follows:

1. **Minimal Handoff Architecture**: A multi-role framework that maintains a single context window, aiming to reduce information loss associated with inter-agent communication. We use "Minimal Handoff" to describe this design goal rather than claim complete elimination.

2. **Aspect-Oriented Orchestration**: Application of AOP to LLM workflows, allowing runtime injection of behaviors like gleaning and persona management without altering core task logic.

3. **Spec-Script Integration**: A mechanism bridging natural language intent with deterministic code execution, ensuring that AI reasoning is grounded in reliable tools.

4. **Cursor Implementation**: A reference implementation within the Cursor IDE, demonstrating practical feasibility for daily software engineering workflows.[1]

Scope and Positioning. This paper presents a methodology contribution rather than an empirical study. We describe the design principles, architectural decisions, and implementation of `cursor-agent-team`, validated through a self-referential implementation experience (Section 6). Quantitative evaluation comparing task success rates or cognitive load against baseline systems is deferred to future work. Our contribution lies in the conceptual framework—minimal handoff design, Aspect-Oriented Prompting, and the Spec-Script loop—which we argue addresses structural limitations in current MAS designs.

The remainder of this paper is organized as follows: Section 2 reviews related work in IA and MAS. Section 3 details our design philosophy. Section 4 presents the technical architecture. Section 5 describes the Cursor-based implementation. Section 6 presents our implementation experience of using the framework to write this paper. Finally, Section 7 discusses implications and limitations, followed by conclusions.

2 Related Work

2.1 Intelligence Augmentation and Human-AI Teaming

The concept of Intelligence Augmentation (IA) centers on systems that expand human cognitive capabilities rather than replacing them [Licklider, 1960]. Recent research emphasizes the shift from passive AI consumption to active collaboration [Jiang et al., 2025], where the human acts as a "conductor" of AI capabilities [National Academies of Sciences, Engineering, and Medicine, 2022]. Wu et al. [2025] highlight that effective teaming requires shared mental models and continuous alignment, which is difficult to achieve when AI systems are opaque or fragmented.

2.2 Multi-Agent Systems and Context Challenges

Multi-Agent Systems (MAS) like AutoGen [Wu et al., 2023] and MetaGPT [Hong et al., 2023] distribute tasks among specialized agents. While scalable, these systems face significant challenges

[1] Source code available at `https://github.com/thiswind/cursor-agent-team`.

in context management. Shekkizhar et al. [2025] identify identity failures where agents lose track of their roles during interaction. The disjoint nature of agent memories can lead to information loss during handoffs, exacerbated by the "Lost in the Middle" phenomenon in long-context models [Liu et al., 2023]. Wan et al. [2025] and Joo et al. [2025] propose mechanisms to enhance long-horizon reasoning, but often retain the multi-instance architecture that fragments the conversation. Zhao et al. [2024] attempts to scale to 128k context via collaboration, but still relies on inter-agent message passing which incurs serialization costs. Our research-first approach in the `/discuss` phase mitigates knowledge cutoffs by integrating retrieval-augmented planning [Qin et al., 2024], ensuring that plans are synthesized from fresh, verified information rather than training data alone.

Table 1 summarizes key architectural differences between `cursor-agent-team` and representative MAS frameworks.

Table 1: Illustrative comparison of agent orchestration approaches. Values are based on documentation review and author experience; independent empirical verification is not provided. See Wu et al. [2023], Hong et al. [2023], and LangGraph documentation for authoritative descriptions.

Feature	**cursor-agent-team**	**AutoGen**	**MetaGPT**	**LangGraph**
Context Model	Single conversation	Inter-agent messages	Shared memory	Graph state
Role Switching	Prompt swap (same instance)	New agent instance	Role-based agents	Node transitions
Human-in-the-loop	Phase-gated (mandatory)	Optional callbacks	Role-assigned	Configurable
Orchestration	AOP-style rules (.mdc)	Python code	SOP documents	Graph definition
Primary Use Case	Single-user IDE workflow	Multi-agent chat	Software team sim	Stateful workflows
Dimensions where other frameworks excel:				
Parallel Execution	Limited (sequential)	Native support	Agent parallelism	Node parallelism
Multi-party Simulation	Not supported	Native support	Native support	Configurable
Backend Flexibility	Single LLM only	Multi-model routing	Multi-model routing	Multi-model routing

This comparison is qualitative; we do not claim superiority across all dimensions. AutoGen and MetaGPT excel at simulating multi-party collaboration, while LangGraph provides fine-grained state control. Our contribution targets the single-user, context-critical scenario where handoff overhead is the primary bottleneck.

Recent approaches like RaDA [Gioacchini et al., 2024] and RPG [Lyu et al., 2024] have demonstrated the efficacy of retrieval-augmented planning. However, these often operate as distinct steps. In our framework, the `/discuss` phase explicitly integrates this retrieval step into the conversational flow, creating a shared epistemic foundation before any plan is committed.

2.3 Orchestration and Formal Constraints

To mitigate agent chaos, orchestration frameworks have emerged. Zhang et al. [2025] propose the Tool-Environment-Agent (TEA) protocol to structure interactions. Similarly, Qiu et al. [2025] advocate for a "Blueprint First" approach where workflows are deterministically defined before execution. Li et al. [2024] introduce formal language constraints to ensure controllable behavior. Our work builds on these by using natural language specifications (Rules) as soft constraints and Python scripts as hard constraints, integrated within a standard IDE workflow. Unlike Daunis [2025] which proposes a new DSL, we use existing Markdown and Python ecosystems to lower the barrier to entry.

2.4 Aspect-Oriented Programming

Aspect-Oriented Programming (AOP) addresses cross-cutting concerns that scatter throughout a codebase [Kiczales et al., 2001, Agrawal and Kaur, 2013]. While traditionally applied to software

engineering, we apply AOP to prompt engineering and agent behavior. By treating concerns like "persona" or "gleaning" as aspects, we maintain a separation of concerns that is lacking in monolithic agent instructions [Coelho et al., 2016].

LangChain's callback system [Chase, 2023] provides a similar mechanism for injecting behavior at specific execution points. However, LangChain callbacks are primarily designed for logging, tracing, and streaming—they do not support the glob-based, context-sensitive rule injection that enables our aspect-oriented approach. Our contribution is the conceptual mapping of AOP terminology (join points, aspects, weaving) to prompt engineering, providing a vocabulary for reasoning about cross-cutting concerns in LLM workflows.

2.5 Persona and Communication in Human-AI Collaboration

The use of personas in conversational AI has gained attention as a method to improve user engagement and communication quality. Jiang et al. [2024] introduce PersonaGym, a framework for evaluating persona consistency in LLM-based agents, revealing that even state-of-the-art models struggle to maintain consistent character traits across extended interactions. Research on *Persona Drift* [Liu et al., 2024] has documented how stylistic elements from previous turns can contaminate subsequent model outputs, leading to gradual identity erosion in multi-turn dialogues.

This body of work motivates our *Persona Sandboxing* approach. Rather than allowing persona traits to permeate the entire reasoning process, we isolate persona expression to a dedicated "Output Layer." The core "Work Layer"—where code generation, analysis, and technical reasoning occur—operates in a persona-free context. A dedicated `history_context_handler` aspect parses conversation history to strip persona artifacts (e.g., honorifics, emojis, colloquial expressions) before they enter the model's reasoning context, aiming to prevent indirect contamination even when previous turns contained persona-styled output. Empirical validation of this isolation mechanism—measuring actual contamination rates under controlled conditions—remains future work.

3 Design Philosophy and Methodology

Our design is grounded in three core pillars that prioritize human cognitive empowerment over autonomous automation.

3.1 Pillar 1: Multi-Role, Not Multi-Agent

We posit that for a single user, a single continuous conversation with a unified AI model is cognitively superior to managing a swarm of disjoint agents. We adopt a "one brain, many hats" approach. The LLM switches roles (e.g., from `/discuss` partner to `/crew` executor) by changing its system prompt context, but the conversation history remains intact. This ensures minimal handoff overhead: the "Writer" role inherently knows what the "Planner" role discussed, because they share the same memory stream.

3.2 Pillar 2: Human-in-the-loop by Design

Following the National Academies' recommendation [National Academies of Sciences, Engineering, and Medicine, 2022], we design for human-AI teaming where the human is the decision-maker. The AI proposes plans, but the human approves them. Critical phases (e.g., moving from drafting to polishing) are gated by human verification. This mitigates the risk of runaway agent loops and ensures the output aligns with user intent. For example, before executing a destructive Git

operation, the system presents a *Preflight Check* summary and waits for explicit user confirmation. Similarly, the transition from `/discuss` to `/crew` is a deliberate human action, acting as a cognitive checkpoint. We formalize this through "Phase Markers" (e.g., `[Phase 1 DONE]`), which serve as mandatory synchronization primitives. The agent cannot proceed to the next phase until the human explicitly validates the current output, creating a verifiable chain of custody for the project's state.

3.3 Pillar 3: Empowerment via Tooling

We aim to democratize expert-level workflows. A user may not know the intricacies of writing a commit message or structuring a research paper, but by invoking a specialized role, they inherit the codified expertise of a PhD-level academic. This represents a transfer of procedural knowledge through the agent interface [Risko and Gilbert, 2023]. By encoding best practices into the agent's persona rules, we effectively "download" a PhD advisor's procedural knowledge into the user's workspace, democratizing access to high-quality academic output.

Our design principles reinforce this empowerment: **Plan-and-Execute** separates strategic intent from tactical execution; **Retrieval-Augmented Planning** ensures decisions are grounded in fresh information; a dedicated **Agent Workspace** provides memory permanence; and **Persona Sandboxing** prevents style from compromising technical accuracy.

4 Architecture: Aspect-Oriented and Spec-Script Integration

Our architecture addresses specific pitfalls in current agent systems through targeted innovations.

4.1 Innovation: Minimal Handoff via Single Conversation

Problem: In MAS, passing a task from a "Researcher" agent to a "Writer" agent involves serializing state and prompting a new instance. This "telephone game" results in context loss. **Solution:** In `cursor-agent-team`, roles are masks worn by the same LLM instance. When the user switches from `/discuss` (planning) to `/crew` (execution), the prompt context is updated (via Cursor Rules), but the chat history persists. The "Writer" can "see" the original debate that led to the plan without explicit summarization.

Defining Handoff. We distinguish between *state-transfer handoff* (serializing context between agent instances, incurring compression loss) and *prompt-swap handoff* (changing only the system prompt while preserving chat history). Our "Minimal Handoff" design aims to eliminate the former; the latter is unavoidable but incurs minimal overhead—typically 1–3KB of rule text versus 50–200KB of compressed conversation state in traditional MAS.[2]

4.2 Innovation: Aspect-Oriented Prompting

Problem: Complex agents require diverse instructions (tone, format, constraints, tooling), leading to prompt pollution and instability. **Solution:** We utilize AOP to weave cross-cutting concerns.

- **Join Points:** User commands (e.g., `/crew`, `/discuss`) serve as join points.

[2] Rule text size estimated from our implementation (sum of loaded `.mdc` files); compressed state size estimated from AutoGen documentation examples and Liu et al. [2023] context compression ratios. These are order-of-magnitude estimates for illustrative comparison.

- **Aspects:** Independent rule files (e.g., `gleaning.mdc`, `tts_speech.mdc`, `persona.mdc`) define diverse behaviors that apply across roles.
- **Weaving:** The Cursor context engine automatically retrieves relevant rules based on the command, effectively weaving aspects into the runtime prompt.

Traditional "Skill" architectures lack this capability. A skill is a flat tool invocation, whereas an aspect is a behavior that modifies the execution flow itself. For example, the `gleaning` aspect intercepts the completion of a task to perform introspection, a behavior that would be awkward to implement as a standalone skill without modifying the core agent loop.

4.3 Innovation: Spec-Script Loop

Problem: LLMs are probabilistic and struggle with deterministic protocols (e.g., "always check file existence before write"). **Solution:** We decouple intent from execution.

1. **Spec (Natural Language):** A Rule file (e.g., `crew_assistant.mdc`) defines *what* should be done and *why*.
2. **Script (Python):** A local script (e.g., `preflight_check.py`) defines *how* it is done reliably.

The LLM reads the Spec, which instructs it to call the Script. The Script executes deterministically and returns a status, which the LLM then interprets. This "Spec-Script Loop" ensures that critical checks (like bibliography verification) are code-enforced rather than hallucinated.

To enforce strict adherence to this loop, we employ *Phase Markers*. These text tokens act as synchronization primitives, confirming that the LLM has completed a mandatory step in the external plan before proceeding. This mechanism mirrors the "automaton-based supervision" in Formal-LLM [Li et al., 2024], but implemented via natural language prompting and script validation rather than a separate control module.

5 Implementation and Platform Selection

We implemented `cursor-agent-team` within the Cursor IDE, a fork of VS Code integrated with LLM capabilities. Cursor was selected for its unique "Rules" feature, which allows context-sensitive injection of instructions, effectively serving as the AOP weaving engine.

For clarity, we define key terms used throughout this section:

- **Rule**: A `.mdc` file containing natural language instructions loaded into the LLM context.
- **Aspect**: A rule that addresses cross-cutting concerns (e.g., persona, gleaning) rather than core task logic.
- **Command**: A user-invoked role switch (e.g., `/discuss`, `/crew`) that triggers rule loading.

This architecture relies on Cursor's rule injection system, which functions as an AOP weaving engine. Rules defined in `.cursor/rules/*.mdc` are loaded at runtime into the LLM's context based on glob patterns and command triggers. This glob-based loading mechanism is the core enabler of our AOP approach. It ensures that rules are not just static instructions but runtime-reactive aspects that react to the user's current activity (the "Join Point"). For instance, the `social_media_policy.mdc` rule is only woven into the context when the agent detects intent to interact with external platforms, ensuring compliance without polluting the context of purely local coding tasks.

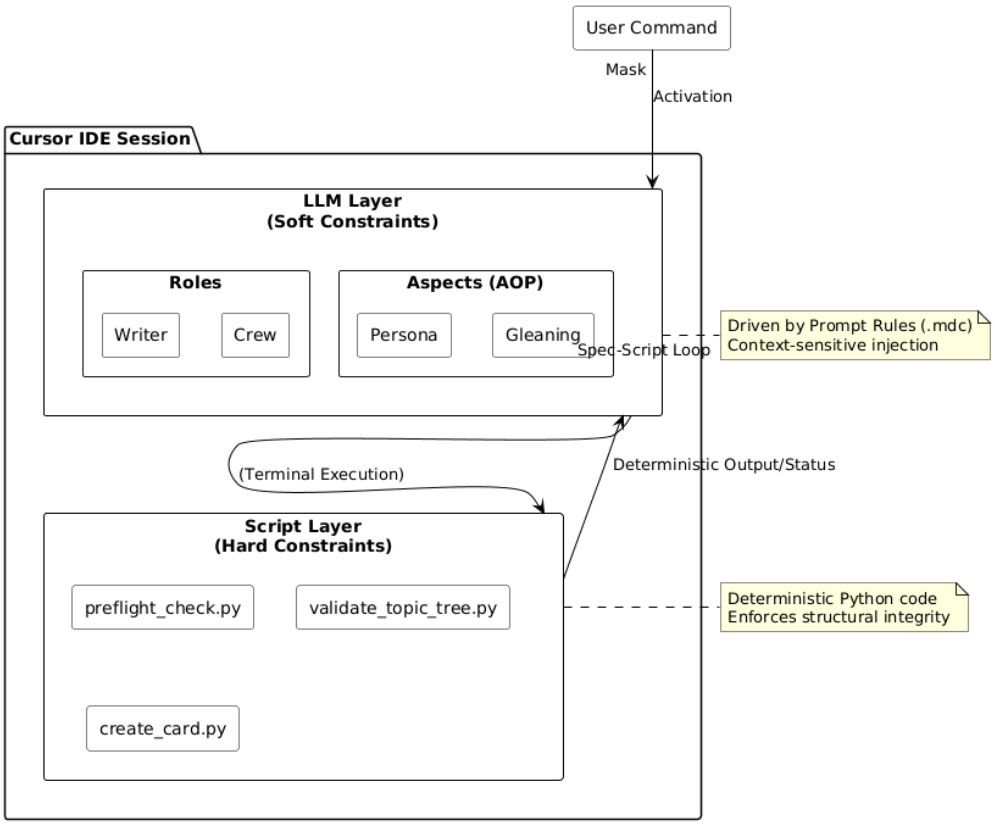

Figure 1: Architecture of cursor-agent-team, illustrating the interaction between the LLM Layer (Roles, Aspects) and the Script Layer (Tools, Verification) within the Cursor IDE context.

5.1 The Script Layer

Beneath the LLM interaction lies a deterministic Python script layer. Key components include:

- `validate_topic_tree.py`: Enforces the structural integrity of the discussion archives.
- `preflight_check.py`: Verifies the environment state before complex operations.
- `create_card.py`: Captures gleaning insights into structured JSON files.

The script layer enforces hard constraints. `validate_topic_tree.py` ensures the discussion history remains a valid tree structure, preventing corruption of the long-term memory. `preflight_check.py` acts as a safety interlock, verifying environmental prerequisites before complex operations begin. `create_card.py` structures unstructured insights into JSON-formatted "cards," enabling systematic knowledge accumulation. These scripts are invoked by the agent via a unified terminal interface, providing a grounding mechanism for the AI's reasoning. The execution flow is strictly controlled by *Phase Markers* (e.g., `[Phase 1 DONE]`), which serve as synchronization primitives between the LLM's internal state and the external plan status.

We also implement *Persona Sandboxing* to maintain professional fidelity. While the agent may use a persona for user interaction, the actual work execution—code generation and analysis—occurs in a "clean room" context stripped of persona traits. We explicitly isolate the "Output Layer" (where persona traits like honorifics and emojis are applied) from the "Work Layer" (where technical reasoning occurs).

The `history_context_handler` aspect implements this isolation through a two-stage process. First, persona-styled outputs in conversation history are wrapped with `<persona_styled>` XML tags at generation time, providing explicit boundary markers. Second, when the model processes history for the work layer, it is instructed to extract only technical content (code blocks, file paths, error messages, numerical data) from tagged regions while ignoring stylistic elements (honorifics, emojis, colloquial particles). This approach relies on prompt-based instruction rather than programmatic filtering, trading implementation simplicity for potential imperfection. The effectiveness of this isolation mechanism has not been empirically validated; measuring actual contamination rates under controlled conditions remains future work.

5.2 The Agent Workspace: Cognitive Artifacts

A persistent challenge in LLM-based agents is the transience of the context window. Once a conversation ends or is summarized, detailed intermediate reasoning is lost. We address this through a dedicated `ai_workspace/` directory that serves as the agent's external memory.

This design draws on three research streams. First, *scratchpad reasoning* [Nye et al., 2021] demonstrates that allowing models to generate intermediate computation steps before producing a final answer improves output quality. Our workspace provides a persistent scratchpad where the agent can record research notes, draft analyses, and intermediate results. Second, work on *external memory* for LLMs, exemplified by MemGPT [Packer et al., 2023], shows that agents benefit from mechanisms to store and retrieve information beyond the context window. Our workspace serves this function, with structured directories for different artifact types. Third, research on *staged generation* [Yang et al., 2025] indicates that iterative refinement through planning outperforms direct output generation. The plan files in our workspace (e.g., `PLAN-AA-001.md`) instantiate this principle: the agent first generates a structured plan, which is then executed step-by-step.

The workspace contains three primary components: `notes/` for persistent scratchpad content and research synthesis; `plans/` for structured execution plans that guide multi-step workflows; and `inspiration_capital/cards/` for "gleaning" artifacts—insights captured during task execution that may prove useful in future, unrelated contexts. This last component enables serendipitous knowledge retrieval: at the start of each `/discuss` session, the agent draws random cards from its accumulated insights, potentially sparking connections that would otherwise be lost.

5.3 Workflow

The standard workflow follows a `discuss` → `plan` → `execute` cycle.

1. **Discuss**: The user invokes `/discuss` to investigate a topic. The agent draws upon "inspiration cards" (randomized context) to stimulate creative divergence.

2. **Plan**: The discussion crystallizes into a structured Markdown plan file (e.g., `PLAN-AA-001.md`).

3. **Execute**: The user invokes `/crew`, which reads the plan and executes it phase-by-phase, ensuring strictly gated progress.

6 Implementation Experience: Writing This Paper

To validate the framework's practical utility, we employed `cursor-agent-team` to write this paper—a form of "dogfooding" that subjects the methodology to its own claims.

6.1 Methodology

The paper was produced through five structured plans (PLAN-AA-001 through PLAN-AA-005), each executed via the `/crew` command. The initial plan (PLAN-AA-001) covered four phases: literature survey, LaTeX skeleton setup, full draft generation, and figure creation. Subsequent plans addressed audit feedback: vocabulary replacement (PLAN-AA-002, PLAN-AA-003), duplicate removal and final polish (PLAN-AA-004), and content expansion (PLAN-AA-005, this section).

Throughout, the `/discuss` phase preceded plan generation, ensuring that the writing approach was grounded in fresh literature searches rather than solely the model's training data. For instance, references to PersonaGym, MemGPT, and staged generation research were retrieved during `/discuss` sessions and subsequently integrated into the plan.

6.2 Observations

Several framework features proved beneficial. The **minimal handoff** architecture allowed the `/crew` role to retain full context of design discussions that occurred during `/discuss`. When expanding the Related Work section, the writer had immediate access to the rationale behind Persona Sandboxing without requiring re-explanation.

Illustrative Token Comparison. During the writing of this paper, a typical role transition (e.g., `/discuss` to `/crew`) involved loading approximately 2,500 tokens of rule text while preserving the full conversation history (averaging 15,000–40,000 tokens across sessions).[3] In contrast, a hypothetical MAS handoff would require compressing this history into a summary prompt—prior work suggests such compression typically retains 10–20% of the original context [Liu et al., 2023]. While we lack controlled experiments, this order-of-magnitude difference illustrates the potential context preservation benefit of our approach.

The **Spec-Script Loop** was exercised repeatedly. The `preflight_check.py` script verified environment state before each plan execution. The `validate_topic_tree.py` script maintained the integrity of the discussion archive as topics evolved. Critically, vocabulary ban enforcement required external verification via `grep` rather than relying on the model's self-correction—a concrete instance of the Spec-Script principle.

Phase Markers enforced workflow discipline. Each plan step required explicit output of markers like `[Phase 2 DONE]`, preventing the agent from skipping steps or conflating phases. When markers were missing, the system flagged the response as invalid.

The **Gleaning aspect** captured several insights during execution, including "Tool State Mismatch Risk" (the danger of edit tools failing when the agent's memory diverges from disk state) and "Negative Constraint Verification" (the necessity of deterministic external checks for vocabulary bans). These cards now reside in the workspace for potential future use.

6.3 Challenges Encountered

Three challenges emerged. First, **tool reliability**: the `StrReplace` tool occasionally failed when the agent's internal model of a file diverged from its actual disk state. The workaround was a "Trust but Verify" pattern—reading the file fresh before edits, and in severe cases, performing full-file rewrites rather than incremental edits.

[3] Token counts were obtained from Cursor IDE session logs during actual writing sessions; values represent order-of-magnitude estimates rather than precise measurements.

Second, **word count versus semantic density**: initial drafts were semantically dense but numerically short of target word counts. Expansion required adding substantive content (e.g., this implementation experience) rather than padding, which the `/crew` role's style guidelines would have flagged.

Third, **vocabulary ban enforcement**: positive instructions ("write professionally") proved insufficient to prevent formulaic phrasing. The solution was explicit negative verification: after each writing phase, running `grep` for banned terms and iterating until zero matches were found. This experience validated the Spec-Script philosophy—deterministic external checks are necessary for constraints that LLMs struggle to self-enforce.

7 Discussion

7.1 Solving the Human-AI Teaming Problem

Our approach addresses the core challenge of Human-AI teaming: maintaining shared context while using AI scale. By keeping the human in the loop as a conductor and using a single conversation thread, we minimize the cognitive load associated with managing multiple agent identities. The Spec-Script integration ensures that the AI's actions remain predictable and safe, fostering trust in the collaboration [Wang and Lu, 2025].

7.2 Generalizability

While the methodology's core principles—AOP prompting, Spec-Script loops, and Multi-Role orchestration—are conceptually transferable, our implementation relies on Cursor-specific features, particularly the glob-based rule injection system. Porting to other IDEs would require equivalent context-injection mechanisms. These patterns can be adapted to any environment that supports runtime system prompt injection and tool execution, such as custom agent runtimes or other AI-enhanced IDEs.

The core innovation lies in the *Command-as-Role* and *Rule-as-Aspect* mapping, which is a transferable design pattern. However, we have not validated portability to other platforms. While VS Code with Copilot and JetBrains AI offer LLM integration, they may lack the glob-based rule injection that enables our AOP approach. Porting would require either platform-specific workarounds or custom middleware to replicate Cursor's context-injection semantics.

7.3 Limitations and Future Work

Our current implementation relies on a single underlying LLM for all roles. While this ensures consistency, it precludes the use of specialized models (e.g., a coding-specific model vs. a creative writing model) within the same context window without external routing. The reliance on the Cursor IDE limits immediate portability to other environments, though the core principles are platform-agnostic. This "Cursor lock-in" is a conscious trade-off for the tight integration that enables our spec-script loop to function reliably.

Self-evaluation bias. The implementation experience (Section 6) uses the framework to evaluate itself, introducing inherent bias: the framework was iteratively refined during the writing process. Independent replication on an external project—ideally by researchers unfamiliar with the system—would provide stronger evidence of the claimed benefits.

Future work will investigate hybrid architectures that can swap backend models while preserving the conversation frontend. For instance, future iterations could route coding tasks to specialized

models while retaining the main conversational model for orchestration, transparent to the user. Additionally, we plan to conduct empirical user studies to quantify the reduction in cognitive load compared to traditional multi-agent chat interfaces.

7.4 Lessons Learned

The implementation experience surfaced several generalizable lessons. **Negative constraint verification**: Positive instructions ("avoid clichés") are insufficient for enforcing vocabulary bans; deterministic external checks (e.g., `grep`) are necessary to mathematically prove compliance. Trusting the LLM's self-correction is less reliable than external validation.

Tool reliability: Edit tools can fail when the agent's internal model of file content diverges from the actual disk state. A "Trust but Verify" pattern—reading fresh before editing, and falling back to full-file rewrites when incremental edits fail—proved essential.

Workflow enforcement: Phase markers function as synchronization primitives, but their effectiveness depends on explicit output format requirements. Without mandatory markers, agents tend to skip steps or conflate phases. The prescription must be structural, not merely advisory.

8 Conclusion

We have presented `cursor-agent-team`, a framework that redefines AI-assisted development through a Multi-Role, Single-Conversation architecture. By prioritizing minimal handoff continuity and using Aspect-Oriented Programming for prompt orchestration, we address the fragmentation and reliability issues observed in current Multi-Agent Systems. Our integration of natural language specifications with deterministic scripts provides a foundation for Intelligence Augmentation, enabling users to orchestrate complex workflows with confidence. The implementation experience of writing this paper demonstrates the framework's practical utility and surfaces lessons—on negative constraint verification, tool reliability, and workflow enforcement—that may generalize to other agent systems. As AI tools evolve, we suggest that such methodology-first, human-centric designs may be essential for realizing the full potential of human-AI symbiosis.

References

Ritu Agrawal and Amardeep Kaur. Review on aspect oriented programming. *International Journal of Advanced Computer Science and Applications*, 4(9), 2013. doi: 10.14569/IJACSA.2013.040904.

Harrison Chase. LangChain: Building applications with LLMs through composability, 2023. URL `https://github.com/langchain-ai/langchain`. GitHub repository.

Roberta Coelho, Awais Rashid, Alessandro Garcia, Fabiano Ferrari, Nélio Cacho, Uirá Kulesza, Arndt von Staa, and Carlos Lucena. Techniques for the identification of crosscutting concerns: A systematic literature review. In *Software Engineering and Advanced Applications*. Springer, 2016. doi: 10.1007/978-3-319-32467-8_50.

Ivan Daunis. A declarative language for building and orchestrating LLM-powered agent workflows. *arXiv preprint arXiv:2512.19769*, 2025. doi: 10.48550/arXiv.2512.19769.

Luca Gioacchini, Carlos García-Olano, Alberto Testoni, Ximing Zhu, Zhaofeng Wang, and Mohit Bansal. RaDA: Retrieval-augmented web agent planning with LLMs. In *Findings of the Associa-*

tion for Computational Linguistics: ACL 2024. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.802.

Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023. doi: 10.48550/arXiv.2308.00352.

Yuxiao Jiang, Yun Hou, Qiao Zheng, Jiang Xie, Linxiao Guan, and Wenlong Liu. PersonaGym: Evaluating persona agents and LLMs. *arXiv preprint arXiv:2407.18416*, 2024. doi: 10.48550/arXiv.2407.18416.

Zhihao Jiang, Yun Wang, Yan Lu, and Zheng Huang. From consumption to collaboration. *arXiv preprint arXiv:2504.02780*, 2025. doi: 10.48550/arXiv.2504.02780. CHI 2025.

Taejong Joo, Shu Ishida, Ivan Sosnovik, Bryan Lim, Sahand Rezaei-Shoshtari, Adam Gaier, and Robert Giaquinto. Graph of agents: Principled long context modeling by emergent multi-agent collaboration. *arXiv preprint arXiv:2509.21848*, 2025. doi: 10.48550/arXiv.2509.21848.

Gregor Kiczales, Erik Hilsdale, Jim Hugunin, Mik Kersten, Jeffrey Palm, and William G. Griswold. An overview of AspectJ. In *Proceedings of the 15th European Conference on Object-Oriented Programming*, ECOOP 2001, pages 327–354. Springer, 2001. doi: 10.1007/3-540-45337-7_18.

Zelong Li, Wenyue Hua, Hao Wang, He Zhu, and Yongfeng Zhang. Formal-LLM: Integrating formal language and natural language for controllable LLM-based agents. *arXiv preprint arXiv:2402.00798*, 2024. doi: 10.48550/arXiv.2402.00798.

J. C. R. Licklider. Man-computer symbiosis. *IRE Transactions on Human Factors in Electronics*, HFE-1:4–11, 1960.

Kenneth Liu, Yingshan Zhu, Zhijing Wang, and Saining Xie. Measuring and controlling persona drift in language model dialogs. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. PMLR, 2024. arXiv:2402.10962.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023. doi: 10.48550/arXiv.2307.03172. Accepted for TACL 2023.

Yuanjie Lyu, Zihan Niu, Zheyong Xie, Chao Zhang, Tong Xu, Yang Wang, and Enhong Chen. Retrieve-plan-generation: An iterative planning and answering framework for knowledge-intensive LLM generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 4683–4702. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.270.

National Academies of Sciences, Engineering, and Medicine. *Human-AI Teaming: State-of-the-Art and Research Needs*. The National Academies Press, Washington, DC, 2022. doi: 10.17226/26355.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Biber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021. doi: 10.48550/arXiv.2112.00114.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. doi: 10.48550/arXiv.2310.08560.

Yiwei Qin, Liang Li, Yufei Luo, Guoyin Zheng, Yongbin Chen, Wenbing Zhang, and Yunbo Huang. Rethinking the bounds of LLM reasoning: Are multi-agent discussions the key? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.331.

Libin Qiu, Yuhang Ye, Zhirong Gao, Xide Zou, Junfu Chen, Ziming Gui, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Kun Zhao. Blueprint first, model second: A framework for deterministic LLM workflow. *arXiv preprint arXiv:2508.02721*, 2025. doi: 10.48550/arXiv.2508.02721.

Evan F. Risko and Sam J. Gilbert. The cognitive architecture of digital externalization. *Educational Psychology Review*, 2023. doi: 10.1007/s10648-023-09818-1.

Sarath Shekkizhar, Romain Cosentino, Adam Earle, and Silvio Savarese. Echoing: Identity failures when LLM agents talk to each other. *arXiv preprint arXiv:2511.09710*, 2025. doi: 10.48550/arXiv.2511.09710.

Guangya Wan, Mingyang Ling, Xiaoqi Ren, Rujun Han, Sheng Li, and Zizhao Zhang. COMPASS: Enhancing agent long-horizon reasoning with evolving context. *arXiv preprint arXiv:2510.08790*, 2025. doi: 10.48550/arXiv.2510.08790.

Yun Wang and Yan Lu. Interaction, process, infrastructure: A unified architecture for human-agent collaboration. *arXiv preprint arXiv:2506.11718*, 2025. doi: 10.48550/arXiv.2506.11718.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. doi: 10.48550/arXiv.2308.08155.

Tongshuang Wu, Gagan Bansal, Eric Horvitz, and Daniel S. Weld. Human-AI collaboration: How AIs augment human teammates. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 5: Tutorial Abstracts)*. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-tutorials.4.

Jinyang Yang, Peixin Zhang, Xuchao Chen, Huan Wang, Fei Ma, Bo Chen, and Shuai Zhao. Thinking before writing: Improving agent writing via iterative argumentation-driven planning. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*. Association for Computational Linguistics, 2025.

Wentao Zhang, Liang Zeng, Yuzhen Xiao, Yongcong Li, Ce Cui, Yilei Zhao, Rui Hu, Yang Liu, Yahui Zhou, and Bo An. AgentOrchestra: Orchestrating multi-agent intelligence with the tool-environment-agent (TEA) protocol. *arXiv preprint arXiv:2506.12508*, 2025. doi: 10.48550/arXiv.2506.12508.

Jun Zhao, Can Zu, Hao Xu, Yi Lu, Wei He, Yiwen Ding, Tao Gui, Qi Zhang, and Xuanjing Huang. LongAgent: Scaling language models to 128k context through multi-agent collaboration. *arXiv preprint arXiv:2402.11550*, 2024. doi: 10.48550/arXiv.2402.11550.